



Bikkad IT Institute

NumPy Fundamentals Notes

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

NumPy Fundamentals Notes

Arrays, Operations, Indexing and Reshaping (Compact Edition)

Prepared for Bikkad IT Institute Students | Level: Beginner to Intermediate | Environment: Jupyter Notebook or Google Colab

Topic Index

- 1. What is NumPy?
- 2. NumPy Arrays vs Python Sequences
- 3. Creating NumPy Arrays
- 4. Array Attributes
- 5. Changing Datatype
- 6. Array Operations
- 7. Array Functions
- 8. Indexing and Slicing
- 9. Iterating
- 10. Reshaping
- 11. Stacking
- 12. Splitting

1. What is NumPy?

NumPy is the fundamental package for scientific and numerical computing in Python. It provides a fast, memory-efficient multidimensional array object called ndarray, along with a large collection of routines for mathematical, logical, and statistical operations, all without writing explicit Python loops.

Syntax

```
import numpy as np
np.array(object)
```

Example 1: Creating 1D, 2D and 3D ndarrays

```
import numpy as np
a = np.array([1, 2, 3])
print(a)
print(a.ndim)

grid = np.array([[1, 2, 3], [4, 5, 6]])
print(grid)
print(grid.ndim)
```

Output / Result

```
[1 2 3]
1
[[1 2 3]
 [4 5 6]]
2
```

Explanation

The first two outputs are the 1D array and its ndim (1); the next two are the 2D array and its ndim (2). np is the usual alias for numpy, and the same np.array() builds 1D, 2D or higher-dimensional arrays depending on how the input list is nested.

Example 2: Built-in mathematical routines

```
import numpy as np
values = np.array([1, 4, 9, 16, 25])
print(values)

sqrt_values = np.sqrt(values)
print(sqrt_values)

total = np.sum(values)
print(total)
```

Output / Result

```
[ 1  4  9 16 25]
[1.  2.  3.  4.  5.]
55
```

Explanation

The first line is the array. The second is np.sqrt applied to every element, and 55 is np.sum of all the values. NumPy ships with many ready-made mathematical functions (ufuncs) that work on the whole array at once.

Example 3: Why loops are avoided - vectorized speed

```
import time
import numpy as np

size = 2_000_000
py_list = list(range(size))
np_arr = np.arange(size)
print(size)

start = time.time()
py_result = [x * 2 for x in py_list]
py_time = time.time() - start
print(round(py_time, 4))

start = time.time()
np_result = np_arr * 2
np_time = time.time() - start
print(round(np_time, 4))
```

Output / Result

```
2000000
0.063
0.0045
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Explanation

2000000 is the array size. The second line is the time (in seconds) for the Python list comprehension, and the last line is the time for `np_arr * 2`. NumPy does the multiplication inside optimized C code (vectorization); exact timings vary by machine, but NumPy is consistently much faster.

2. NumPy Arrays vs Python Sequences

A NumPy ndarray looks similar to a Python list but behaves very differently underneath - in fixed size, in how it stores data types, and in memory usage.

Syntax

```
list.append(value) # Python list - grows dynamically
np.append(array, value) # NumPy array - returns a NEW array
```

Example 1: Fixed size vs dynamic size

```
import numpy as np
py_list = [1, 2, 3]
py_list.append(4)
print(py_list)

arr = np.array([1, 2, 3])
print(arr)

new_arr = np.append(arr, 4)
print(arr)
print(new_arr)
```

Output / Result

```
[1, 2, 3, 4]
[1 2 3]
[1 2 3]
[1 2 3 4]
```

Explanation

The first line shows the list after `append()` - it grew in place to 4 items. The next two lines show `arr` is still `[1 2 3]` even after `np.append()`, and the last line is the brand-new array `np.append()` returned. An ndarray has a fixed size, so appending always creates a new array.

Example 2: Homogeneous data type vs mixed types

```
import numpy as np
mixed_list = [1, 'two', 3.0, True]
print(mixed_list)

types = [type(x).__name__ for x in mixed_list]
print(types)

mixed_arr = np.array([1, 'two', 3.0, True])
print(mixed_arr)
print(mixed_arr.dtype)
```

OUR LOCATIONS

Output / Result

```
[1, 'two', 3.0, True]
['int', 'str', 'float', 'bool']
['1' 'two' '3.0' 'True']
<U32
```

Explanation

The first line is the mixed list and the second shows each item keeps its own type (int, str, float, bool). The NumPy array needs one type for all elements, so it turns everything into text - the last line <U32 means 'Unicode string up to 32 characters'.

Example 3: Memory footprint comparison

```
import sys
import numpy as np
py_list = list(range(1000))
np_arr = np.arange(1000)

list_bytes = sys.getsizeof(py_list) + sum(sys.getsizeof(x) for x in py_list)
print(list_bytes)

np_bytes = np_arr.nbytes
print(np_bytes)
```

Output / Result

```
36056
8000
```

Explanation

36056 is the memory used by the Python list plus all its integer objects; 8000 is the memory of the NumPy array holding the same 1000 numbers (1000 x 8 bytes). Because every element has the same fixed-width type stored side by side, NumPy uses far less memory.

3. Creating NumPy Arrays

NumPy offers several constructors for building arrays: converting existing Python sequences, generating regular numeric ranges, and creating arrays pre-filled with a constant or random values.

Syntax

```
np.array(data, dtype=None)
np.arange(start, stop, step)
np.ones(shape); np.zeros(shape); np.identity(n)
np.random.random(shape); np.linspace(start, stop, num)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 1: np.array with dtype control

```
import numpy as np
a = np.array([1, 2, 3])
print(a)

b = np.array([[1, 2], [3, 4]])
print(b)

float_arr = np.array([1, 2, 3], dtype=float)
print(float_arr)
print(float_arr.dtype)
```

Output / Result

```
[1 2 3]
[[1 2]
 [3 4]]
[1. 2. 3.]
float64
```

Explanation

The nesting of the input list decides the number of dimensions, and the dtype parameter overrides NumPy's automatic type inference for every element.

Example 2: Ranges with arange and reshape

```
import numpy as np
seq = np.arange(1, 11, 2)
print(seq)

grid = np.arange(16).reshape(2, 2, 2, 2)
print(grid.shape)
```

Output / Result

```
[1 3 5 7 9]
(2, 2, 2, 2)
```

Explanation

np.arange(start, stop, step) generates values like Python's range(), and .reshape() rearranges that flat sequence into any shape with a matching total element count.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 3: Pre-filled and random arrays

```
import numpy as np
ones_arr = np.ones((2, 3))
print(ones_arr)

identity_arr = np.identity(3)
print(identity_arr)

np.random.seed(42)
random_arr = np.random.random((2, 3))
rounded_random = np.round(random_arr, 2)
print(rounded_random)

lin = np.linspace(-10, 10, 5, dtype=int)
print(lin)
```

Output / Result

```
[[1. 1. 1.]
 [1. 1. 1.]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
[[0.37 0.95 0.73]
 [0.6  0.16 0.16]]
[-10 -5  0  5 10]
```

Explanation

The outputs in order: a 2x3 array of ones, a 3x3 identity matrix (1s on the diagonal), 2x3 random values between 0 and 1 (rounded to 2 decimals), and 5 evenly spaced whole numbers from -10 to 10 made by np.linspace.

4. Array Attributes

Every ndarray carries metadata describing its shape and storage: ndim, shape, size, itemsize, and dtype. These are essential for debugging and for writing code that adapts to the data it receives.

Syntax

```
a.ndim; a.shape; a.size; a.itemsize; a.dtype
```

Example 1: ndim and shape

```
import numpy as np
a1 = np.arange(10, dtype=np.int32)
print(a1)
print(a1.ndim)
print(a1.shape)

a2 = np.arange(12, dtype=float).reshape(3, 4)
print(a2)
print(a2.ndim)
print(a2.shape)

a3 = np.arange(8).reshape(2, 2, 2)
print(a3)
print(a3.ndim)
print(a3.shape)
```

Output / Result

```
[0 1 2 3 4 5 6 7 8 9]
1
(10,)
[[ 0.  1.  2.  3.]
 [ 4.  5.  6.  7.]
 [ 8.  9. 10. 11.]]
2
(3, 4)
[[[0 1]
  [2 3]]

 [[4 5]
  [6 7]]]
3
(2, 2, 2)
```

Explanation

Each array is printed followed by its ndim and shape: a1 has 1 axis with shape (10,), a2 has 2 axes with shape (3, 4), and a3 has 3 axes with shape (2, 2, 2). shape lists the length along each axis, in order.

Example 2: size and itemsize

```
import numpy as np
a1 = np.arange(10, dtype=np.int32)
a2 = np.arange(12, dtype=float).reshape(3, 4)
a3 = np.arange(8).reshape(2, 2, 2)

print(a1.size)
print(a1.itemsize)
print(a2.size)
print(a2.itemsize)
print(a3.size)
print(a3.itemsize)
```

OUR LOCATIONS

Output / Result

10
4
12
8
8
8

Explanation

The lines come in pairs - size then itemsize - for a1, a2 and a3: a1 has 10 elements of 4 bytes (int32), a2 has 12 elements of 8 bytes (float64), and a3 has 8 elements of 8 bytes (int64). size is the total element count; itemsize is the bytes used by each element.

Example 3: dtype - the element data type

```
import numpy as np
a1 = np.arange(10, dtype=np.int32)
a2 = np.arange(12, dtype=float).reshape(3, 4)
a3 = np.arange(8).reshape(2, 2, 2)

print(a1.dtype)
print(a2.dtype)
print(a3.dtype)
```

Output / Result

int32
float64
int64

Explanation

The three lines are the dtypes of a1, a2 and a3. a1 was created as int32, a2 as float (float64), and a3 falls back to NumPy's default integer type, int64.

5. Changing Datatype

Once an array exists, its data type can be converted with .astype(), useful for controlling memory usage or preparing data for a function that expects a specific type.

Syntax

```
a.astype(new_dtype)
```

Example 1: Float <-> integer conversion

```
import numpy as np
prices = np.array([10.9, 20.2, 30.8])
print(prices)

whole = prices.astype(int)
print(whole)

back_to_float = whole.astype(np.float64)
print(back_to_float)
```

Output / Result

```
[10.9 20.2 30.8]
[10 20 30]
[10. 20. 30.]
```

Explanation

Converting float to int truncates the decimal part (it does not round); converting back to float simply adds the decimal point again.

Example 2: Numbers to strings

```
import numpy as np
ids = np.array([101, 102, 103])
print(ids)

as_text = ids.astype(str)
print(as_text)
print(as_text.dtype)
```

Output / Result

```
[101 102 103]
['101' '102' '103']
<U21
```

Explanation

The second line shows each number converted to its text form, and <U21 means the array now holds Unicode strings. `astype(str)` is handy before joining numbers into labels or file names.

Example 3: Numbers to boolean

```
import numpy as np
flags = np.array([0, 5, -3, 0, 1])
print(flags)

as_bool = flags.astype(bool)
print(as_bool)
```

Output / Result

```
[ 0  5 -3  0  1]
[False  True  True False  True]
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Explanation

When converting to bool, zero becomes False and any non-zero number, positive or negative, becomes True.

6. Array Operations

NumPy overloads Python's arithmetic and comparison operators so they work element-wise. A scalar operation applies the same value to every element, while a vector operation combines two arrays position by position.

Syntax

```
a + scalar; a ** scalar; a == value; a > value
a1 + a2; a1 * a2; a1 ** a2
```

Example 1: Scalar arithmetic operations

```
import numpy as np
a1 = np.arange(12).reshape(3, 4)
print(a1)

squared = a1 ** 2
print(squared)
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  4  9]
 [16 25 36 49]
 [64 81 100 121]]
```

Explanation

Every element of a1 is independently raised to the power of 2; the operator is broadcast across the whole array with no explicit loop.

Example 2: Scalar and vector relational operations

```
import numpy as np
a1 = np.arange(12).reshape(3, 4)
a2 = np.arange(12, 24).reshape(3, 4)
print(a1)
print(a2)

eq_check = a2 == 15
print(eq_check)

gt_check = a1 > 5
print(gt_check)

lt_check = a1 < a2
print(lt_check)
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
[[False False False  True]
 [False False False False]
 [False False False False]]
[[False False False False]
 [False False  True  True]
 [ True  True  True  True]]
[[ True  True  True  True]
 [ True  True  True  True]
 [ True  True  True  True]]
```

Explanation

Comparison operators return a Boolean array of the same shape - against a scalar, or against a second array of matching shape, marking True wherever the condition holds.

Example 3: Vector arithmetic between two arrays

```
import numpy as np
a1 = np.arange(12).reshape(3, 4)
a2 = np.arange(12, 24).reshape(3, 4)

sum_arr = a1 + a2
print(sum_arr)

prod_arr = a1 * a2
print(prod_arr)

small_a = np.array([[1, 2], [3, 4]])
small_b = np.array([[3, 1], [2, 2]])
print(small_a)
print(small_b)

power_arr = small_a ** small_b
print(power_arr)
```

Output / Result

```
[[12 14 16 18]
 [20 22 24 26]
 [28 30 32 34]]
[[ 0  13  28  45]
 [ 64  85 108 133]
 [160 189 220 253]]
[[1 2]
 [3 4]]
[[3 1]
 [2 2]]
[[ 1  2]
 [ 9 16]]
```

Explanation

When two arrays share the same shape, arithmetic operators combine them element by element. Keeping the numbers small for the power example also avoids silent integer overflow from large exponents.

OUR LOCATIONS

7. Array Functions

Beyond basic operators, NumPy ships with aggregation functions (sum, mean, max...), mathematical ufuncs (sin, log, exp...), and linear-algebra helpers such as the dot product, all built to run efficiently across whole arrays.

Syntax

```
np.sum(a, axis=None); np.mean(a); np.std(a)
np.sin(a); np.log(a); np.exp(a)
np.dot(a, b)
```

Example 1: Aggregations and statistical summaries

```
import numpy as np
np.random.seed(1)
a1 = np.round(np.random.random((3, 3)) * 100)
print(a1)

col_sums = np.sum(a1, axis=0)
print(col_sums)

row_means = np.round(np.mean(a1, axis=1), 2)
print(row_means)

std_val = round(np.std(a1), 2)
print(std_val)
```

Output / Result

[[42. 72. 0.]
[30. 15. 9.]
[19. 35. 40.]]
[91. 122. 49.]
[38. 18. 31.33]
20.31

Explanation

The first output is the 3x3 array. The second line is the sum of each column (axis=0), the third is the mean of each row (axis=1), and 20.31 is the standard deviation of the whole array. axis=0 collapses down the rows, axis=1 across the columns, and no axis reduces everything to a single value.

Example 2: Trigonometric, log, exp and rounding

```
import numpy as np
vals = np.array([1, 2.3, 4.7, 8])
print(vals)

sin_vals = np.round(np.sin(vals), 2)
print(sin_vals)

log_vals = np.round(np.log(vals), 2)
print(log_vals)

floor_vals = np.floor(vals)
print(floor_vals)

ceil_vals = np.ceil(vals)
print(ceil_vals)
```

Output / Result

```
[1.  2.3 4.7 8. ]
[ 0.84  0.75 -1.    0.99]
[0.    0.83 1.55 2.08]
[1.  2.  4.  8.]
[1.  3.  5.  8.]
```

Explanation

After the array itself, the lines are: sine of each value, natural log of each value, floor (always rounds down) and ceil (always rounds up). Every function is applied to each element independently.

Example 3: Dot product and matrix multiplication

```
import numpy as np
a2 = np.arange(12).reshape(3, 4)
a3 = np.arange(12, 24).reshape(4, 3)
print(a2)
print(a3)

dot_result = np.dot(a2, a3)
print(dot_result)
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[12 13 14]
 [15 16 17]
 [18 19 20]
 [21 22 23]]
[[114 120 126]
 [378 400 422]
 [642 680 718]]
```

Explanation

`np.dot()` performs true matrix multiplication for 2D arrays: the inner dimensions (4 and 4 here) must match, and the result takes the outer shape (3 x 3).

OUR LOCATIONS

8. Indexing and Slicing

NumPy uses zero-based indexing, exactly like Python lists. For multidimensional arrays, a single index expression can select an axis-by-axis position, and slices follow the familiar start:stop:step pattern with stop excluded.

Syntax

```
a[row, column] or a[axis0, axis1, axis2]  
a[start:stop:step]
```

Example 1: 1D and 2D indexing

```
import numpy as np  
a1 = np.arange(10)  
print(a1)  
  
a2 = np.arange(12).reshape(3, 4)  
print(a2)  
  
print(a1[0])  
print(a1[-1])  
print(a1[5])  
print(a2[1, 0])  
print(a2[2, 3])
```

Output / Result

```
[0 1 2 3 4 5 6 7 8 9]  
[[ 0  1  2  3]  
 [ 4  5  6  7]  
 [ 8  9 10 11]]  
0  
9  
5  
4  
11
```

Explanation

After the two arrays, the five numbers are $a1[0] = 0$ (first), $a1[-1] = 9$ (last), $a1[5] = 5$, $a2[1, 0] = 4$ (row 1, column 0) and $a2[2, 3] = 11$ (row 2, column 3). $a2[\text{row}, \text{column}]$ picks one element of a 2D array directly.

Example 2: 3D indexing

```
import numpy as np  
a3 = np.arange(8).reshape(2, 2, 2)  
print(a3)  
  
print(a3[1, 0, 1])  
print(a3[1, 1, 0])
```

OUR LOCATIONS

Output / Result

```
[[[0 1]
  [2 3]]

  [[4 5]
  [6 7]]]
5
6
```

Explanation

a3[1, 0, 1] is 5 and a3[1, 1, 0] is 6. A 3D array needs three indices: the first selects the block, the second the row inside it, and the third the column inside that row.

Example 3: 1D and 2D slicing patterns

```
import numpy as np
a1 = np.arange(10)
a2 = np.arange(12).reshape(3, 4)
print(a1)
print(a2)

slice1 = a1[2:8:2]
print(slice1)

reversed_a1 = a1[::-1]
print(reversed_a1)

slice2 = a2[0:2, 1::2]
print(slice2)

slice3 = a2[:, 1::2]
print(slice3)
```

Output / Result

```
[0 1 2 3 4 5 6 7 8 9]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[2 4 6]
[9 8 7 6 5 4 3 2 1 0]
[[1 3]
 [5 7]]
[[ 1  3]
 [ 9 11]]
```

Explanation

start:stop:step works as with Python lists, with stop never included and a step of -1 reversing the array. Each axis of a 2D array can be sliced independently in one bracket, separated by a comma.

9. Iterating

NumPy arrays can be looped over directly, but what each iteration returns depends on the number of dimensions. For a fully flat, element-by-element walk regardless of shape, np.nditer is the dedicated tool.

Syntax

```
for x in a: ...  
for x in np.nditer(a): ...
```

Example 1: Iterating 1D and 2D arrays

```
import numpy as np  
a1 = np.arange(5)  
print(a1)  
for i in a1:  
    print(i, end=' ')  
print()  
  
a2 = np.arange(12).reshape(3, 4)  
print(a2)  
for row in a2:  
    print(row)
```

Output / Result

```
[0 1 2 3 4]  
0 1 2 3 4  
[[ 0  1  2  3]  
 [ 4  5  6  7]  
 [ 8  9 10 11]]  
[0 1 2 3]  
[4 5 6 7]  
[ 8  9 10 11]
```

Explanation

Looping over a 1D array yields one scalar at a time; looping over a 2D array yields one full row (a 1D sub-array) per iteration, not one scalar element.

Example 2: Iterating a 3D array

```
import numpy as np  
a3 = np.arange(8).reshape(2, 2, 2)  
print(a3)  
for block in a3:  
    print(block)
```

Output / Result

```
[[[0 1]  
 [2 3]]  
  
 [[4 5]  
 [6 7]]]  
[[[0 1]  
 [2 3]]  
 [[4 5]  
 [6 7]]]
```

Explanation

Looping over a 3D array yields one entire 2D block per iteration; the pattern generalizes so each loop peels off the outermost axis.

OUR LOCATIONS

Example 3: Flattened iteration with np.nditer

```
import numpy as np
a3 = np.arange(8).reshape(2, 2, 2)
print(a3)
for value in np.nditer(a3):
    print(value, end=' ')
print()
```

Output / Result

```
[[[0 1]
  [2 3]]

  [[4 5]
  [6 7]]]
0 1 2 3 4 5 6 7
```

Explanation

np.nditer visits every individual scalar value inside the array in order, regardless of how many dimensions it has - useful when the shape doesn't matter.

10. Reshaping

Reshaping changes how an array's elements are arranged into axes without changing the underlying data or the total number of elements. Transpose and ravel are two of the most common reshaping tools.

Syntax

```
a.reshape(new_shape); a.reshape(3, -1)
a.T or np.transpose(a)
a.ravel(); a.flatten()
```

Example 1: Reshape and transpose

```
import numpy as np
a = np.arange(1, 7)
print(a)

m = a.reshape(2, 3)
print(m)

transposed = m.T
print(transposed)
```

Output / Result

```
[1 2 3 4 5 6]
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

OUR LOCATIONS

Explanation

reshape() rearranges the same six values into the requested 2 x 3 grid, and transposing that grid flips rows and columns to produce a 3 x 2 array.

Example 2: Ravel and auto-sized reshape with -1

```
import numpy as np
a3 = np.arange(8).reshape(2, 2, 2)
print(a3)

raveled = a3.ravel()
print(raveled)

b = np.arange(12)
print(b)

reshaped_b = b.reshape(3, -1)
print(reshaped_b)
```

Output / Result

```
[[[0 1]
   [2 3]]

  [[4 5]
   [6 7]]]
[0 1 2 3 4 5 6 7]
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Explanation

ravel() collapses an array of any dimension into a single flat 1D sequence. Passing -1 for one dimension in reshape tells NumPy to compute that size automatically.

Example 3: ravel (view) vs flatten (copy)

```
import numpy as np
m = np.array([[1, 2], [3, 4]])
print(m)

r = m.ravel()
print(r)

r[0] = 99
print(m)
```

Output / Result

```
[[1 2]
 [3 4]]
[1 2 3 4]
[[99  2]
 [ 3  4]]
```

Explanation

The second line is `r = m.ravel()`. After `r[0] = 99`, printing `m` shows 99 in its first position - `ravel()` returned a view that shares memory with `m`, so editing `r` changed `m`. `flatten()` would instead return an independent copy and leave `m` unchanged.

11. Stacking

Stacking joins two or more arrays together into a single larger array. NumPy provides direction-specific helpers for the common cases, plus a general-purpose function for stacking along a brand-new axis.

Syntax

```
np.hstack((a, b)); np.vstack((a, b))
np.stack((a, b), axis=0); np.concatenate((a, b), axis=0)
```

Example 1: Horizontal and vertical stacking

```
import numpy as np
a4 = np.arange(12).reshape(3, 4)
a5 = np.arange(12, 24).reshape(3, 4)
print(a4)
print(a5)

hstacked = np.hstack((a4, a5))
print(hstacked)

vstacked = np.vstack((a4, a5))
print(vstacked)
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
[[ 0  1  2  3 12 13 14 15]
 [ 4  5  6  7 16 17 18 19]
 [ 8  9 10 11 20 21 22 23]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
```

Explanation

`hstack` joins arrays side by side along the columns (same row count required); `vstack` joins them top to bottom along the rows (same column count required).

Example 2: Stacking along a brand-new axis

```
import numpy as np
b1 = np.array([1, 2, 3])
b2 = np.array([4, 5, 6])
print(b1)
print(b2)

stacked_axis0 = np.stack((b1, b2), axis=0)
print(stacked_axis0)

stacked_axis1 = np.stack((b1, b2), axis=1)
print(stacked_axis1)
```

Output / Result

```
[1 2 3]
[4 5 6]
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

Explanation

`np.stack()` introduces a completely new axis to hold the joined arrays, unlike `hstack/vstack` which join along an existing axis.

Example 3: General-purpose concatenate

```
import numpy as np
a4 = np.arange(12).reshape(3, 4)
a5 = np.arange(12, 24).reshape(3, 4)
print(a4)
print(a5)

concat_axis0 = np.concatenate((a4, a5), axis=0)
print(concat_axis0)

concat_axis1 = np.concatenate((a4, a5), axis=1)
print(concat_axis1.shape)
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
(3, 8)
```

Explanation

np.concatenate() is the general form behind hstack and vstack: axis=0 behaves like vstack and axis=1 behaves like hstack for 2D arrays.

12. Splitting

Splitting is the reverse of stacking: it breaks one array into several smaller arrays, either into equal sections or at custom boundaries that you specify.

Syntax

```
np.hsplit(a, sections); np.vsplit(a, sections)
np.array_split(a, sections); np.split(a, n, axis=k)
```

Example 1: Horizontal and vertical split

```
import numpy as np
a4 = np.arange(20).reshape(4, 5)
print(a4)

parts = np.hsplit(a4, 5)
print(parts[0].ravel())
print(parts[1].ravel())

a5 = np.arange(12, 24).reshape(3, 4)
print(a5)

top, bottom = np.vsplit(a5, [1])
print(top)
print(bottom)
```

Output / Result

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
[ 0  5 10 15]
[ 1  6 11 16]
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
[[12 13 14 15]]
[[16 17 18 19]
 [20 21 22 23]]
```

Explanation

After a4, the next two lines are the first two of the five column pieces from np.hsplit, flattened with ravel(). After a5, vsplit(a5, [1]) splits at row 1: top has only the first row and bottom has the remaining two rows.

Example 2: Uneven splits with array_split

```
import numpy as np
a1 = np.arange(10)
print(a1)

parts = np.array_split(a1, 3)
for part in parts:
    print(part)
```

Output / Result

```
[0 1 2 3 4 5 6 7 8 9]
[0 1 2 3]
[4 5 6]
[7 8 9]
```

Explanation

array_split() tolerates a section count that does not divide the array evenly, distributing the remaining elements across the earliest sections.

Example 3: Splitting a 3D array along a chosen axis

```
import numpy as np
a3 = np.arange(24).reshape(2, 3, 4)
print(a3.shape)

left, right = np.split(a3, 2, axis=2)
print(left.shape)
print(right.shape)
```

Output / Result

```
(2, 3, 4)
(2, 3, 2)
(2, 3, 2)
```

Explanation

np.split() works on any axis of an n-dimensional array; here axis=2 splits the last (depth) axis in half instead of the default first axis.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune